



Коллектив исслед... ·

 Избранное

ТЕХНОЛОГИЯ +

Программирование +

Искусственный Интеллект +

Наука о данных +

Машинное обучение +

The Durable AI Runtime: полное руководство

Уровень отказоустойчивости с открытым исходным кодом, который восстанавливает рабочие нагрузки после сбоев инфраструктуры и логики.

8 минут чтения · 5 дней назад



Paolo Perrone



Слушай



Поделиться

Среда выполнения ИИ — это уровень реализации, который позволяет перенести рабочие процессы ИИ, машинного обучения и агентных систем с ноутбука в производственную среду. Она управляет состоянием выполнения, восстанавливается после сбоев инфраструктуры, таких как нехватка памяти, обеспечивает устойчивость длительных заданий и позволяет выполнять обучение и логический вывод в одной системе. Flyte 2 от Union.ai — первая среда выполнения ИИ с открытым исходным кодом, выпущенная в общий доступ 4 августа 2026 года.

Работает в блокноте, но не проходит проверку. Ошибка нехватки памяти в 3 часа ночи приводит к сбою почти завершенного обучающего прогона, и повторная попытка начинается с нуля. Отладка в итоге занимает больше времени, чем сборка.

Стандартное решение, которое я вижу у более чем 1 млн своих читателей, — это все та же «изолента»: (1) оркестратор, созданный для статичных пайплайнов, (2) логика повторных попыток, которой никто полностью не доверяет, и (3) отсутствие возможности автоматического восстановления после сбоев, вызванных инфраструктурой.

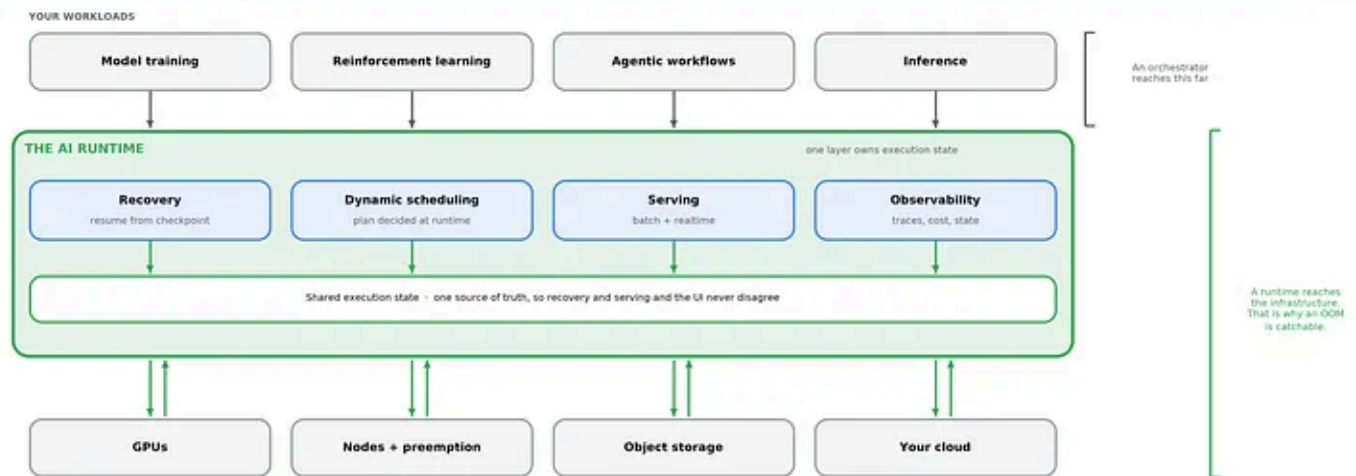
У уровня долговечности, который нужно исправить, наконец-то появилось название. Он называется средой выполнения ИИ, и его первая реализация с открытым исходным кодом появилась 4 августа.

Что на самом деле представляет собой среда выполнения Durable AI

Ненавижу, когда люди слишком усложняют категории инфраструктуры, поэтому вот простая версия.

Среда выполнения ИИ — это уровень реализации стека ИИ: часть, которая превращает ИИ, машинное обучение или агентный рабочий процесс из того, что работает на вашем ноутбуке, в то, что может использоваться в продакшене. Устойчивая среда выполнения — это среда, которая может видеть вашу инфраструктуру и самостоятельно восстанавливаться после сбоев, вызванных инфраструктурой.

Ваша модель отвечает за интеллектуальную составляющую. Среда выполнения отвечает за все, что может выйти из строя: (1) восстановление после сбоя кода или инфраструктуры в процессе выполнения задания, (2) надежность заданий, которые выполняются часами или днями, (3) агентские рабочие процессы, которые динамически адаптируются во время выполнения.



Независимо от того, занимаетесь ли вы обучением моделей, обучением с подкреплением, пакетным выводом или выводом в реальном времени и т. д., именно среда выполнения позволяет вам перейти к продакшену.

Этот уровень внезапно стал важным, потому что рабочие процессы с использованием ИИ перестали быть конвейерами. Сегодня они динамически разветвляются, циклически повторяются, вызывают инструменты, обрабатывают мультимодальные данные и при этом тратят часы работы на дорогостоящих графических процессорах (петли агентов — самый яркий пример). Все традиционные оркестраторы предполагают, что вы знаете весь план до запуска первой задачи. Рабочий процесс, который сам определяет свой следующий шаг во время выполнения (как агент), нарушает это предположение.

Почему ваш оркестратор продолжает проигрывать

Традиционный оркестратор требует, чтобы весь граф был готов заранее. Вы передаете ему DAG, и он выполняет именно этот DAG. Такой подход подходит для ночного ETL. Но он не работает, когда вам нужно, чтобы рабочий процесс самостоятельно решал, куда двигаться дальше: агент, который выбирает следующий инструмент на основе того, что вернул предыдущий, задание обучения, которое разветвляется в зависимости от метрики проверки, исследовательский пайплайн, который разветвляется в зависимости от того, что он находит. (Я видел, как читатели сталкивались с этой проблемой при создании своих первых агентов. Это самая распространенная причина провала в почему большинство агентов ИИ терпят неудачу в продакшене.)

And when something dies at 3am? The orchestrator's usual answer is the whole run again, from the top, on GPUs you pay for by the minute.

The pain compounds in three specific ways, and if you run AI workloads you have probably met all of them: (1) infrastructure failures like OOM, node interruptions, and container preemption at worst are totally invisible to you and at best need to be solved manually, (2) iteration is slow because run-break-debug-restart is a manual loop, (3) brittle workflows run up compute costs because a long job that dies restarts from zero, and you pay again for every GPU-hour it already burned.

None of this is a model problem.

(The most expensive failures engineers describe to me are never the model being wrong; they're the long run that died just short of done.)

The stack teams duct-tape instead

So teams build the missing layer themselves, one incident at a time.

A scheduler here. Custom retry decorators there. A checkpointing script someone wrote after the last OOM. A separate serving stack for inference, because the orchestrator was never meant to serve anything. A dashboard bolted over the top that can see the tasks but never the state.

Every seam between those systems is a place where state gets lost. The orchestrator does its retry without knowing a checkpoint exists. The serving stack has no idea the model behind it just got retrained. You end up debugging the glue more often than the workload (which is exactly the work a runtime exists to delete).

A runtime's whole argument is that one layer should own execution state, so that recovery, inference, and observability all read from the same source of truth. That's the one place you'd actually check at 3am.

Flyte 2: the first open-source AI runtime

On August 4, Union.ai shipped Flyte 2, the first open-source AI runtime to scale durable AI, ML, and agentic workloads on your own infrastructure.

Flyte itself is well entrenched in engineering teams. The project has tens of millions of downloads and 4K+ organizations running it, including frontier AI labs and Fortune 500 teams, and it's developed alongside partners like NVIDIA, Amazon,

Google, and Nebius. Flyte 2 takes that same track record and rebuilds it specifically to do the job just described, now landing its full open-source GA.

Three capabilities do the work:

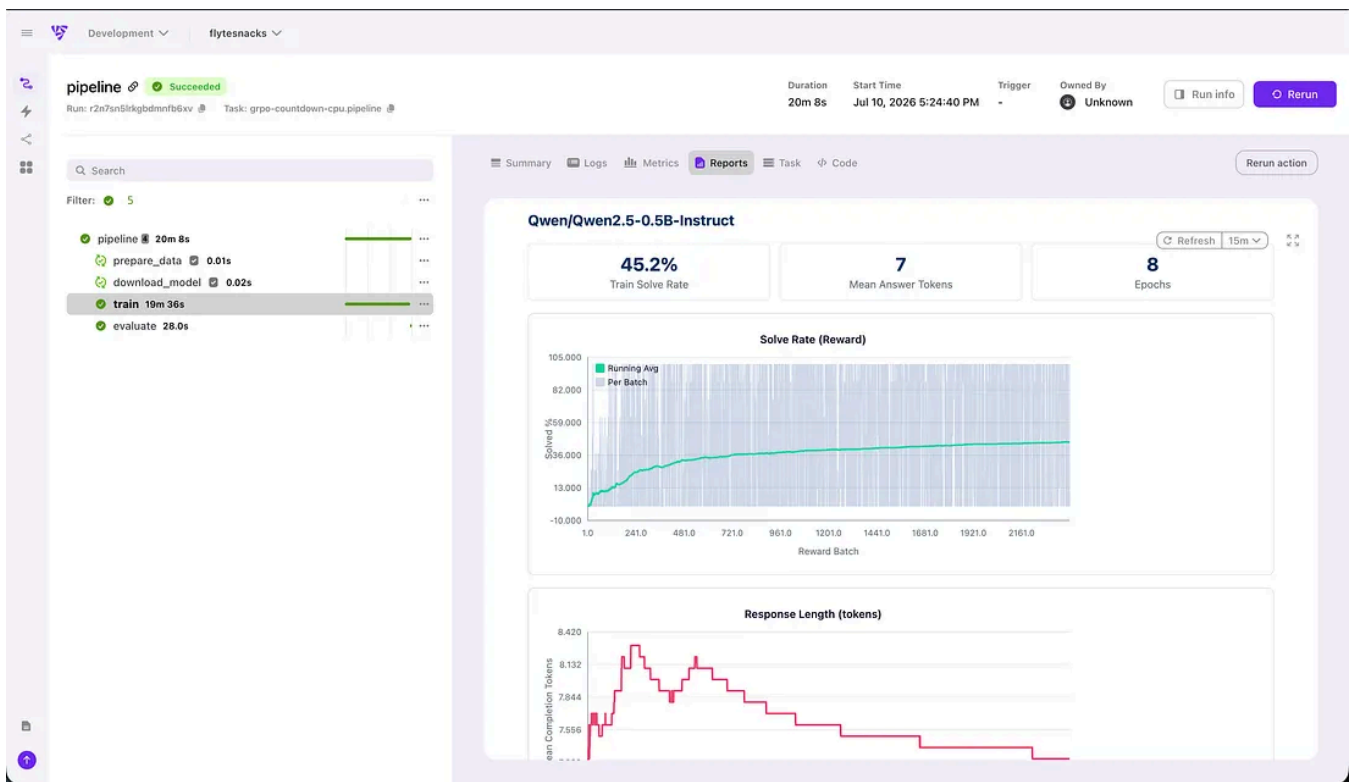
1. **Durable, self-healing workflows for infra and code failures.** This is the one that made me sit up. Flyte 2 is infrastructure-aware: an OOM crash is something your code can catch and handle, with the ability to provision the needed resources at runtime. That means you can build workflows that self-heal at runtime from both logic and infra failures.
2. **Dynamic, agent-native execution.** Workflows branch, loop, call tools, and adapt while they run without losing durability. The plan is decided at runtime, by the workflow itself, so agent loops are a native shape instead of a hack on top of a fixed graph.
3. **Real-time and batch inference, same runtime.** Training and serving (whether batch or realtime) run on one system, so the separate serving stack (and the glue code it requires) goes away.

Around those three: a production-level UI out of the box, free to run (the backend and SDK are Apache 2.0, the UI is source-available under a Union license), and everything runs on your machines and your cloud, so your data never leaves.

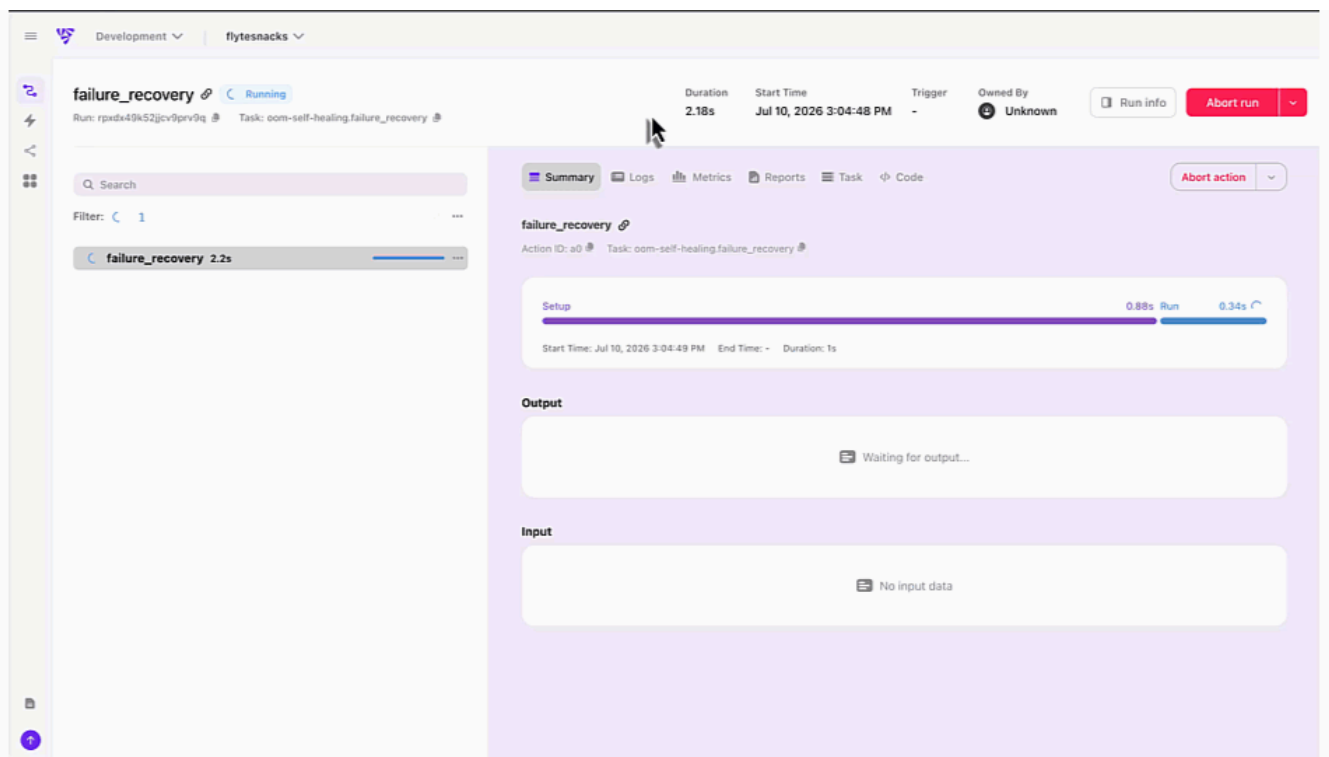
What it looks like when you run it

Union.ai ships reproducible projects with the launch, published in [their examples repo](#). Together, they walk through everything above, and I'd point you at these before any docs page.

[The LLM post training GRPO](#) project is a good one to get started with. It uses the full stack in a single run to use Group Relative Policy Optimization (GRPO), a reinforcement learning algorithm introduced by DeepSeek to fine-tune an LLM to get better performance on a specific task.

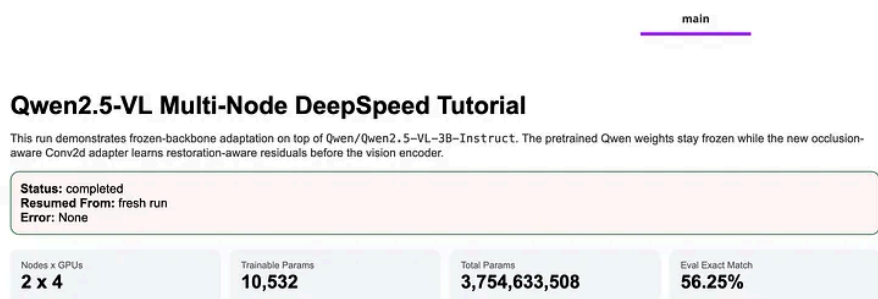


A task fails (the red dot in the UI), then recovers and succeeds on retry instead of restarting the whole run.



The Claude agent research pipeline shows agent-native execution: give it a prompt that returns several research directions and the orchestration adapts at runtime, fanning out instead of following a fixed script.

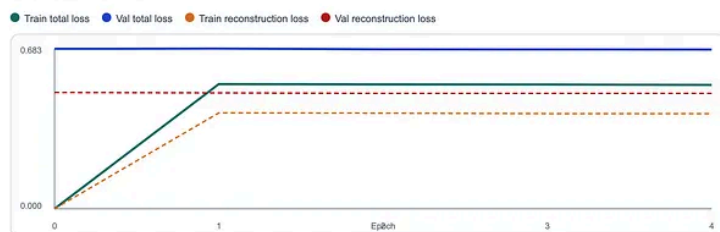
The **VLM app** runs a vision-language model in a pre-built environment, live, on the same runtime that handles the batch work. That's the no-separate-serving-stack story, running.



What This Run Shows

Dataset prep is isolated as a v2 task.
Training scales with Elastic plus DeepSpeed across multiple nodes.
The pretrained backbone is frozen while only the inserted adapter is optimized.
Evaluation renders the final report directly from task artifacts.

Training Curves



The fastest on-ramp is the Flyte 2 devbox, which spins up locally in about five minutes, and Flyte 2 runs locally today.

Everything above is runnable on your own machine before you commit to anything.

Keep your skepticism calibrated

Three things.

First, judge a runtime on durability and let the feature list come second. The question is whether a workflow survives infrastructure failure instead of dying at 3am, because recovery from infra and logic failures is the gap AI teams actually live with every week.

Second, if your workloads are genuinely linear and deterministic, a nightly batch pipeline that never branches, your orchestrator is fine. A runtime earns its place when workflows get dynamic and jobs get long and expensive. Adopting one for a cron job is overkill.

Third, adopting any new execution layer costs a migration, and a fresh GA means you're early. Early has real upside here (Apache 2.0, a large existing Flyte

community on GitHub and Slack), and it's still a line item you should budget honestly.

No layer deletes the hard parts of production. The runtime gives them an owner.

Where this lands

The reframe I'd leave you with: production-readiness in AI has been an execution-layer property all along. We kept treating it as glue code each team writes for itself (mine included, back when I shipped my first agents), which is why every team's glue looks like everyone else's postmortem.

Once the execution layer is a thing you adopt instead of a thing you write, the failure stories change shape. The 3am OOM is a log line now.

My inbox will tell me if that holds.

Given who's already running Flyte in production, I like the odds.

Try it yourself: spin up the Flyte 2 devbox and run the GRPO project. Five minutes to a running local runtime is a cheap way to form your own opinion.

If you're earlier in the journey and agents are the thing breaking on you, start with why most AI agents fail in production and AI agents in 5 levels of difficulty.

If this guide saved you a postmortem, give it 50 claps. It tells Medium to show it to more engineers before their next 3am page.

I break down the engineering behind AI systems like this every week in The AI Engineer, where software engineers get dangerously good at AI engineering. Subscribe free.

FAQ

What is an AI runtime?

An AI runtime is the execution layer that takes AI, ML, and agentic workflows from experiment to production. Most important is that it delivers durable execution across both logic and infrastructure. It manages dynamic workflow logic, keeps long-running jobs durable, recovers from code and infrastructure failures like OOM errors, and supports both real-time and batch inference on your own infrastructure.

Is an AI runtime just an orchestrator with retries?

An orchestrator executes a plan it was handed upfront. A runtime owns execution while it happens: the workflow can change its plan mid-run, failures recover from checkpoints with infrastructure awareness, and inference runs on the same layer. Retries are one feature; owning the execution state is the category.

When should I choose an AI runtime?

If your workloads are linear, deterministic, and short, your current orchestrator is probably enough. If you're running agentic or compute-intensive AI/ML jobs, a runtime like Flyte 2 becomes critical to scaling those workloads durably.

Is Flyte 2 open source, and can I run it locally?

Yes on both. Flyte 2 is Apache 2.0 licensed, runs on your own infrastructure so data never leaves your environment, and runs locally today. The devbox at flyte.org gets you a local environment in about five minutes.

Technology +

Programming +

Artificial Intelligence +

Data Science +

Machine Learning +



Published in Data Science Collective

945K followers · Last published 3 days ago

Advice, insights, and ideas from the Medium data science community



Written by Paolo Perrone

13.6K followers · 172 following

Shipping Production AI: Agents, Inference, GPU. Read by 1M+ AI engineers. Founding Editor Data Science Collective <https://medium.com/data-science-collective>

